

Requerimientos de datos para gráficas de Frecuencia Cardíaca

Objetivo

Este documento describe qué datos necesita el frontend para que funcionen las gráficas del módulo FrecuenciaCardiaca, qué propiedades espera cada componente y por qué son necesarias.

La intención es compartir este contrato con backend para alinear los endpoints y reducir lógica de simulación en frontend.

Componentes involucrados

- GraficaFrecuenciaCardiacaDiaria
- GraficaPromedioFrecuencia
- GraficaReposoActividad
- GraficaDistribucion

Estado actual del backend

En la documentación de Liber Salus existe el tipo de medición Frecuencia cardíaca con `id_measure_type = 19`.

Endpoints relevantes encontrados:

- GET `/mediciones/{id_person}/{id_tipo}/datos-crudos`
- GET `/mediciones/{id_person}/{id_tipo}/serie`
- GET `/mediciones/{id_person}/{id_tipo}/registros`
- GET `/graficas/frecuencia?id_person=...`

Observación importante:

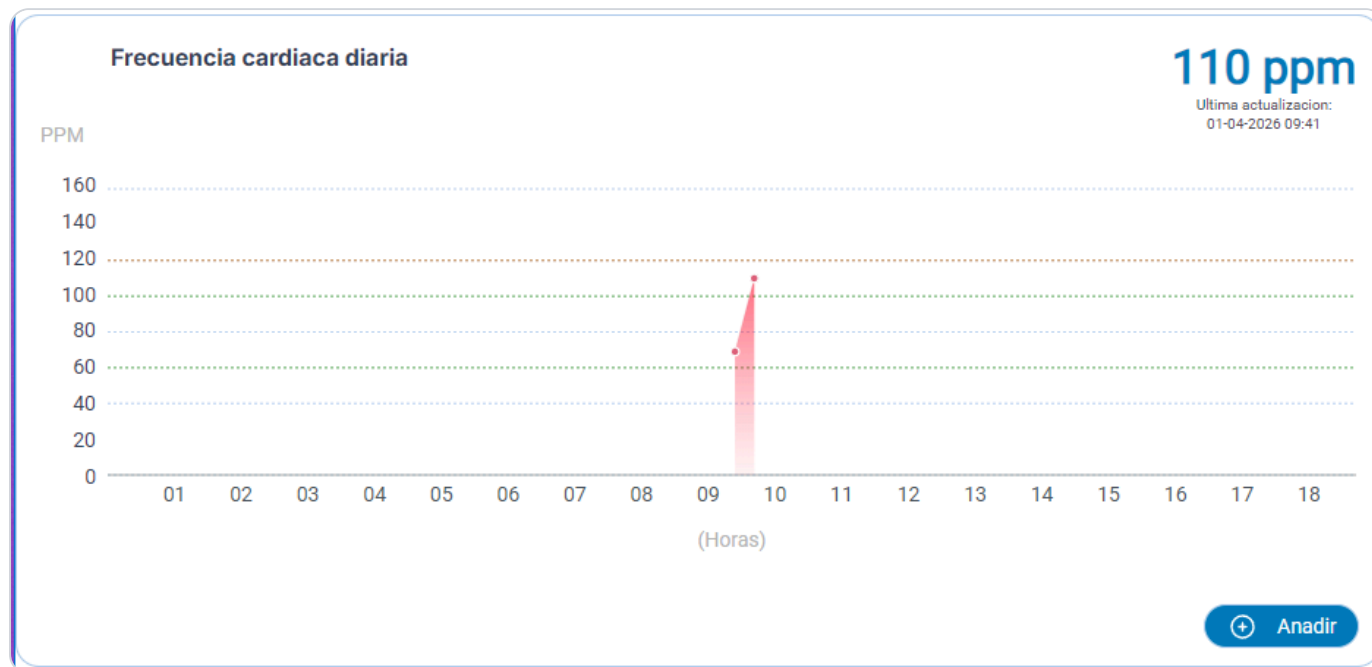
- GET `/graficas/frecuencia` devuelve `text/html`, por lo que no sirve como fuente directa para nuestras gráficas en React.
- Los endpoints tipo `datos-crudos`, `serie` y `registros` sí pueden servir para construir parte de las gráficas.
- Hoy backend no expone de forma explícita un resumen diario con `fcReposo` y `fcActividad`, que sí sería ideal para algunas gráficas.

Resumen de necesidades por gráfica

1. Gráfica diaria

Componente:

- GraficaFrecuenciaCardiacaDiaria.jsx



GraficaDiaria

Propósito:

- Mostrar lecturas puntuales de frecuencia cardíaca a lo largo de un día.
- Mostrar el valor destacado actual.
- Mostrar la última actualización.

Datos mínimos requeridos

```
{
  "pacienteId": "pac-001",
  "fecha": "2026-03-12",
  "ultimaActualizacionISO": "2026-03-12T15:30:00Z",
  "registros": [
    {
      "id": "reg-001",
      "fechaHoraISO": "2026-03-12T06:10:00Z",
      "ppm": 62,
      "tipoRegistro": "reposo"
    },
    {
```

```
{
  "id": "reg-002",
  "fechaHoraISO": "2026-03-12T08:45:00Z",
  "ppm": 104,
  "tipoRegistro": "actividad"
}
```

Propiedades requeridas y justificación

Justificación: identifica a qué persona pertenecen los datos y evita mezclar registros entre pacientes.

- `pacienteId`

Justificación: delimita el día que debe pintarse en la gráfica diaria.

- `fecha`

Justificación: se usa para mostrar la leyenda de "Última actualización" sin tener que inferirla manualmente.

- `ultimaActualizacionISO`

Justificación: es la colección base de lecturas puntuales que alimenta la serie.

- `registros`

Justificación: sirve para trazabilidad, claves estables y futuras operaciones como edición o eliminación.

- `registros[].id`

Justificación: permite ubicar cada lectura en el eje X por hora y minuto.

- `registros[].fechaHoraISO`

Justificación: es el valor principal que se grafica en el eje Y.

- `registros[].ppm`

Justificación: hoy no es indispensable para pintar la línea diaria, pero sí es importante para clasificar lecturas, distinguir captura en reposo o actividad y soportar agregaciones futuras.

- `registros[].tipoRegistro`

Endpoint backend recomendado

Ideal:

- `GET /mediciones/{id_person}/19/registros?fecha=YYYY-MM-DD`

Si backend no filtra por fecha:

- GET /mediciones/{id_person}/19/registros

y frontend filtra el día.

Si se usa el endpoint actual de backend

La respuesta más cercana hoy es:

```
{
  "nombre_medicion": "Frecuencia cardíaca",
  "unidad_medida": "ppm",
  "registros": [
    {
      "id": 101,
      "fecha": "2026-03-04T08:30:00",
      "valor": 120.0,
      "notas": "Toma en ayunas",
      "creado_en": "2026-03-04T08:32:11",
      "actualizado_en": "2026-03-04T08:32:11",
      "eliminado_logico": false
    }
  ]
}
```

Mapeo a frontend:

- registros[].id -> id
- registros[].fecha -> fechaHoraISO
- registros[].valor -> ppm
- registros[].notas -> opcionalmente puede ayudar a derivar tipoRegistro si backend lo guarda allí, aunque no es lo ideal

2. Gráfica de promedio de frecuencia

Componente:

- GraficaPropemidoFrecuencia.jsx



GraficaPromedios

Propósito:

- Mostrar rangos de frecuencia por semana, mes y año.
- Pintar mínimo y máximo por período visible.

Datos mínimos requeridos

Esta gráfica puede construirse desde historial crudo.

```
{
  "pacienteId": "pac-001",
  "fechaInicio": "2026-01-01",
  "fechaFin": "2026-03-31",
  "registros": [
    {
      "id": "hist-001",
      "fechaHoraISO": "2026-03-10T06:00:00Z",
      "ppm": 58,
      "tipoRegistro": "reposo"
    },
    {
      "id": "hist-002",
      "fechaHoraISO": "2026-03-10T13:20:00Z",
      "ppm": 118,
      "tipoRegistro": "actividad"
    }
  ]
}
```

```
}  
]  
}
```

Propiedades requeridas y justificación

Justificación: delimita el rango temporal disponible para filtros y consultas.

- `fechaInicio`

Justificación: delimita el final del historial y permite construir filtros consistentes.

- `fechaFin`

Justificación: permite agrupar lecturas por día, semana, mes o año.

- `registros[].fechaHoraISO`

Justificación: de aquí se calculan mínimos y máximos.

- `registros[].ppm`

Justificación: no es estrictamente necesario para la gráfica promedio actual, pero mantiene consistencia del contrato y puede servir para segmentar en el futuro.

- `registros[].tipoRegistro`

Qué calcula frontend actualmente

Con el historial crudo, frontend genera:

- opciones de filtro por semana
- opciones de filtro por mes
- opciones de filtro por año
- mínimos y máximos por día
- promedio mensual y anual de esos rangos

Endpoint backend recomendado

Puede usarse:

- `GET /mediciones/{id_person}/19/datos-crudos`

o

- `GET /mediciones/{id_person}/19/registros`

Idealmente con filtros de rango:

- GET /mediciones/{id_person}/19/datos-crudos?desde=YYYY-MM-DD&hasta=YYYY-MM-DD

3. Gráfica comparativa Reposo vs Actividad

Componente:

- GraficaReposoActividad.jsx



GraficaRA

Propósito:

- Mostrar dos series en paralelo:
- frecuencia cardíaca en reposo
- frecuencia cardíaca en actividad
- Permitir visualización por semana, mes y año.

Datos mínimos requeridos

Esta gráfica necesita datos agregados por día.

```
{
  "pacienteId": "pac-001",
  "fechaInicio": "2026-03-01",
  "fechaFin": "2026-03-31",
```

```
"registros": [  
  {  
    "fecha": "2026-03-10",  
    "fcReposo": 62,  
    "fcActividad": 116  
  },  
  {  
    "fecha": "2026-03-11",  
    "fcReposo": 65,  
    "fcActividad": 124  
  }  
]
```

Propiedades requeridas y justificación

Justificación: se usa para ubicar cada punto en el período correcto y agrupar por semana, mes o año.

- `registros[].fecha`

Justificación: alimenta la línea azul de reposo.

- `registros[].fcReposo`

Justificación: alimenta la línea naranja de actividad.

- `registros[].fcActividad`

Regla importante

- `fcActividad >= fcReposo`

Justificación: clínicamente y visualmente se asume que la frecuencia en actividad no debería ser menor a la de reposo para el resumen diario mostrado.

Situación actual

Backend hoy no expone esta estructura de forma directa en la documentación revisada.

Si solo se cuenta con mediciones crudas, frontend tendría que inferir:

- `fcReposo` = mínimo del día
- `fcActividad` = máximo del día

Eso puede funcionar como aproximación técnica, pero no necesariamente representa una clasificación clínica real.

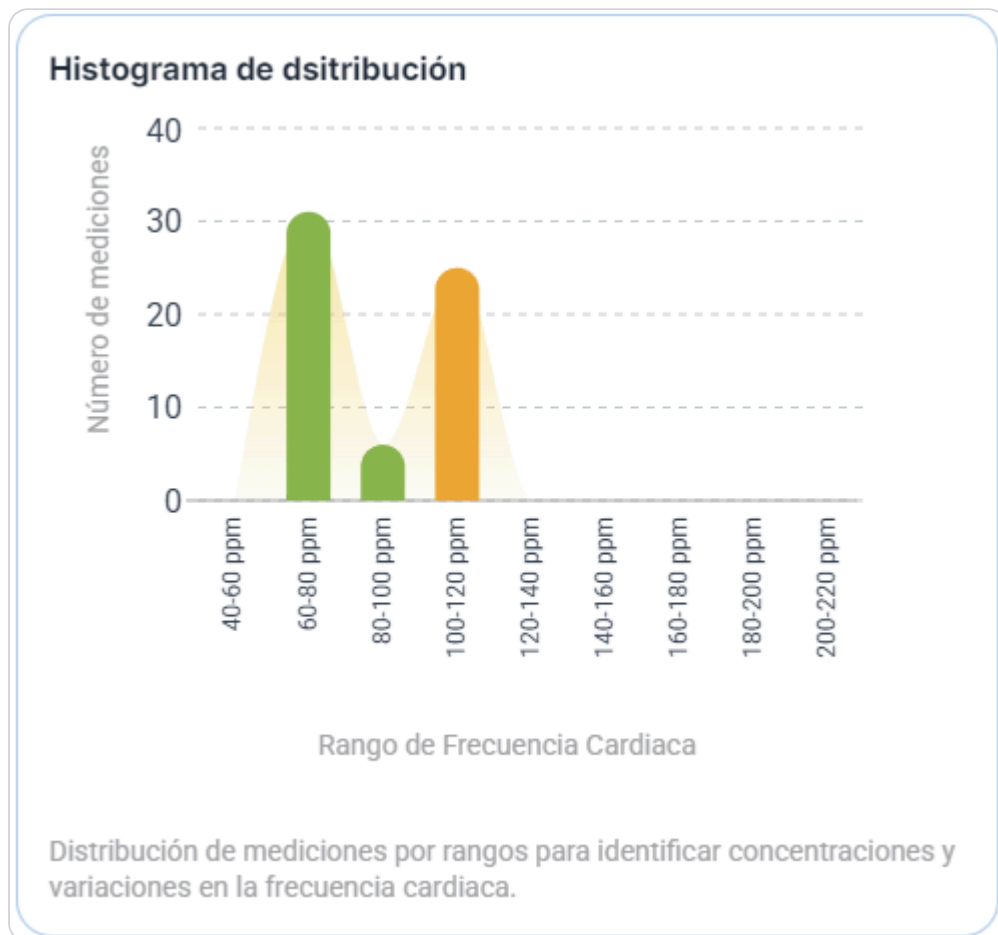
Endpoint backend ideal

- GET /mediciones/{id_person}/19/resumen-diario

Respuesta:

```
[
  {
    "fecha": "2026-03-10",
    "fcReposo": 62,
    "fcActividad": 116
  }
]
```

4. Histograma de distribución



GraficaDistribución

Componente:

- GraficaDistribucion.jsx

Propósito:

- Mostrar cuántas mediciones caen en cada rango de frecuencia.

- Detectar concentración y dispersión de lecturas.

Datos mínimos requeridos

La utilidad actual usa esta estructura:

```
[
  {
    "fecha": "2026-03-10",
    "fcReposo": 62,
    "fcActividad": 116
  },
  {
    "fecha": "2026-03-11",
    "fcReposo": 65,
    "fcActividad": 124
  }
]
```

Propiedades requeridas y justificación

Justificación: permite filtrar por semana, mes o año.

- fecha

Justificación: cuenta como una lectura dentro del histograma.

- fcReposo

Justificación: cuenta como una lectura dentro del histograma.

- fcActividad

Cómo trabaja hoy el frontend

La utilidad `distribucionFrecuencia.utils.js` toma ambos valores de cada día:

- agrega fcReposo
- agrega fcActividad
- cuenta en qué rango cae cada uno

Rangos actuales:

- 40-60 ppm
- 60-80 ppm
- 80-100 ppm
- 100-120 ppm

- 120-140 ppm
- 140-160 ppm
- 160-180 ppm
- 180-200 ppm
- 200-220 ppm

Endpoint backend ideal

Puede reutilizar el mismo resumen diario:

- GET /mediciones/{id_person}/19/resumen-diario

o bien un endpoint ya agregado específicamente para distribución:

- GET /mediciones/{id_person}/19/distribucion

Pero el resumen diario ya es suficiente para este frontend.

Contrato recomendado para backend

Si se quiere cubrir las 4 gráficas sin ambigüedad, backend debería exponer al menos estos dos contratos:

A. Lecturas crudas

Para gráfica diaria y promedio.

```
{
  "pacienteId": "pac-001",
  "registros": [
    {
      "id": "101",
      "fechaHoraISO": "2026-03-10T08:30:00Z",
      "ppm": 120,
      "tipoRegistro": "actividad"
    }
  ]
}
```

B. Resumen diario

Para comparativa e histograma.

```
{
  "pacienteId": "pac-001",
```

```
"registros": [  
  {  
    "fecha": "2026-03-10",  
    "fcReposo": 62,  
    "fcActividad": 116  
  }  
]  
}
```

Recomendación final

Con el backend actual ya es posible conectar:

- GraficaFrecuenciaCardiacaDiaria
- GraficaPromedioFrecuencia

Con los endpoints de registros o datos crudos de Frecuencia cardíaca (`id_tipo = 19`).

Para conectar correctamente:

- GraficaReposoActividad
- GraficaDistribucion

se recomienda pedir a backend un resumen diario explícito con:

- fecha
- fcReposo
- fcActividad

Esto evita inferencias ambiguas desde frontend y deja el contrato mucho más claro, estable y mantenible.